# Ders 2 Genişletilmiş Notu: Tümleyen Gösterimler ve IEEE 754; Sinyal ve Sistem Kavramı, Frekans Düzlemi ve Sinyallerin Sınıflandırılması

## Ders 2 Genişletilmiş Notu: Tümleyen Gösterimler ve IEEE 754; Sinyal ve Sistem Kavramı, Frekans Düzlemi ve Sinyallerin Sınıflandırılması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Tam Sayıların Bilgisayarda Gösterimi
- 2. Gerçel Sayılar ve IEEE 754
- 3. İkili Kodlama: BCD, Gray, Eşlik, ASCII ve Unicode
- 4. Mantıksal İşlemler
- 5. MATLAB'a Kısa Giriş ve Ek Kaynak
- 6. Sinyal ve Sistem Kavramı
- 7. Sinyal ve Sistemlerin İlk Sınıflandırması
- 8. Haberleşme ve Kontrol Sistemleri, Tacoma Köprüsü
- 9. Frekans Düzlemi Neden Var?
- 10. Geri Beslemeli Hız Kontrolü ve Salınım
- 11. Kosinüs Sinyali ve Bileşenleri
- 12. Sinyallerin Sınıflandırılması
- 13. Örnek: Bir Sinyalin Özellikleri ve MATLAB ile Çizimi
- 14. Zaman ve Uzay Fonksiyonu Olarak Sinyaller, Sistem Türleri
- 15. Notasyon ve Örnekleme
- 16. Sinyal İşlemleri: Genlikte ve Zamanda Ölçekleme


## 1. Tam Sayıların Bilgisayarda Gösterimi

Hoca derse, önceki dersten kalan sayı gösterimleriyle başladı. Bu konuların başka derslerde de ayrıntılı görüleceğini, ama ileride lazım olabileceği için burada da gösterdiğini söyledi.

### 1.1 İşaretsiz Tam Sayılar

İşaretsiz ikili tam sayıların (unsigned binary integer) iki özelliği vardır: tam sayıdırlar ve işaretleri yoktur. Bunları göstermek için fazla bir şey yapmak gerekmez, doğrudan taban aritmetiği uygulanır. Sayı sürekli ikiye bölünebilir ya da doğrudan ikilik açılımı yazılabilir.

Onluk sistemde birler, onlar, yüzler basamağı varsa, ikilik sistemde birler, ikiler, dörtler, sekizler, on altılar, otuz ikiler, altmış dörtler, yüz yirmi sekizler basamağı vardır. Onluk sistemde 368 dediğimizde yüzler basamağında 3 tane değer olduğunu anlarız. İkilik sistemde ise tane sayısı yoktur, yalnızca varlık veya yokluk vardır: Basamaktaki değer ya vardır (1) ya yoktur (0).

**Örnek: 127'nin ikilik karşılığı.** 127'yi göstermek için 128'ler basamağı kullanılmaz, o basamak 0 kalır. Sonra yukarıdan aşağı inilir:

- 64'ler basamağı dolar. $127 - 64 = 63$ kalır.
- 32'ler dolar. $63 - 32 = 31$ kalır (ara adım eklendi).
- Aynı şekilde 16, 8, 4, 2 ve 1'ler basamakları da dolar.

Sonuç: $127 = 64 + 32 + 16 + 8 + 4 + 2 + 1$, yani 8 bitte `01111111`. Bu kadar basit olmasının sebebi sayının işaretsiz olmasıdır.

### 1.2 İşaret-Büyüklük Yöntemi

İşaretli tam sayılar daha meşakkatlidir ve üzerine çok kafa yorulmuştur. Önceki mantığı doğrudan sürdürmenin adı işaret-büyüklük yöntemidir (sign magnitude method). $N$ bitin 1 biti işarete ayrılır. Hangi bitin ayrılacağı sorusuna hoca şu cevabı verdi: Sağdaki de olabilir, soldaki de. Arada bir bit de seçilebilir, ama o zaman fazladan işlem gerekir. Literatürde genellikle en soldaki bit kullanılır. Bu bite en yüksek öncelikli bit (most significant bit, MSB), en sağdakine en düşük öncelikli bit (least significant bit, LSB) denir. Kural şudur:

- İşaret biti 1 ise sayı negatif, 0 ise pozitiftir.
- Kalan bitler büyüklüğü (magnitude) saklar.

Hoca 4 bitlik bütün sayıları hızlı yazmanın yolunu da gösterdi. Sağdan birinci sütun birer birer, ikinci sütun ikişer ikişer, üçüncü sütun dörder dörder, dördüncü sütun sekizer sekizer değişir. Soldaki biti işaret sayınca şu tablo çıkar:

| Bit dizisi | Değer | Bit dizisi | Değer |
|---|---|---|---|
| `0000` | $+0$ | `1000` | $-0$ |
| `0001` | $+1$ | `1001` | $-1$ |
| `0010` | $+2$ | `1010` | $-2$ |
| `0011` | $+3$ | `1011` | $-3$ |
| `0100` | $+4$ | `1100` | $-4$ |
| `0101` | $+5$ | `1101` | $-5$ |
| `0110` | $+6$ | `1110` | $-6$ |
| `0111` | $+7$ | `1111` | $-7$ |

Görünüşte her şey güzeldir, ama iki sorun vardır. Birincisi $+0$ ve $-0$ diye iki sıfır bulunur. İkincisi toplama doğrudan çalışmaz.

**Örnek:** $+3 + (-3)$**.** $+3$ = `0011`, $-3$ = `1011`.

- Sağdan: $1 + 1 = 2$, yazılan 0, elde 1.
- $1 + 1 + 1 = 3$, yazılan 1, elde 1.
- $0 + 0 + 1 = 1$.
- $0 + 1 = 1$.

Sonuç `1110`. İşaret biti 1, büyüklük $4 + 2 = 6$, yani $-6$. Oysa 0 bulmamız gerekiyordu. Bu yöntemde matematiksel işlemler için ekstra kontroller gerekir, bu yüzden çok verimli değildir.

### 1.3 Bire Tümleyen

Alternatif olarak bire tümleyen (one's complement) önerilmiştir. Hoca bunun da günümüzün standardı olmadığını söyledi. Pozitif sayılarda işaret-büyüklükle fark yoktur: İlk bit 0'dır, geri kalanı sayının ikilik karşılığıdır. Negatif sayılar ise pozitif sayının bitleri ters çevrilerek (inversion) bulunur: 0'lar 1, 1'ler 0 olur. Yani bir negatif sayıyı yazmak için önce pozitifini bilmek gerekir. Dönüşüm iki yönde de çalışır.

**Örnek:** $-8$ **(8 bit).** $+8$ = `00001000`. Bitler ters çevrilince $-8$ = `11110111` bulunur. İkisi toplanınca `11111111` çıkar, ve bunun sıfıra karşılık gelmesi gerekir. Aynı iki sıfır sorunu burada da vardır.

Hoca bunu 3 bit üzerinde gösterdi:

| Pozitif | Değer | Negatif | Değer |
|---|---|---|---|
| `000` | $+0$ | `111` | $-0$ |
| `001` | $+1$ | `110` | $-1$ |
| `010` | $+2$ | `101` | $-2$ |
| `011` | $+3$ | `100` | $-3$ |

**Örnek:** $+3 + (-3)$ **(3 bit).** `011 + 100 = 111`. Sonuç $-0$'dır. Yöntem işe yarıyor, ama iki sıfırın durumuna dikkat etmek gerekir. Bu da işlemci tarafında ekstra işlem gücü demektir.

### 1.4 İkiye Tümleyen

Sıfırdaki belirsizliği gidermek için yöntem geliştirilmiş ve ikiye tümleyen (two's complement) ortaya çıkmıştır. Günümüzün standardı budur ve tek bir sıfırı vardır. Pozitif sayılar yine işaret-büyüklükteki gibidir. Negatif sayı için iki yol vardır:

- **Birinci yol:** Bitleri ters çevir (bire tümleyen), sonuca 1 ekle. Burada 1 eklemek, yalnız en sağ bitte 1, diğer bitlerde 0 olan sayıyı eklemektir.
- **İkinci yol (kısa yol):** Sağdan sola doğru, karşılaşılan ilk 1 dahil, bütün bitleri aynen kopyala. Sonra kalan bitleri ters çevir.

**Örnek:** $-20$ **(8 bit).** Önce $+20$ bulunur. İlk bit 0'dır, 64'ler ve 32'ler basamağı 0, 16'lar basamağı 1 olur. $20 - 16 = 4$ kalır, 4'ler basamağı da 1 olur. Yani $+20$ = `00010100`.

- Birinci yol: Ters çevrilince `11101011`. Bir eklenince `11101100`.
- İkinci yol: Sağdan ilk 1'e kadar `100` aynen kalır. Kalan `00010` ters çevrilip `11101` olur. Sonuç yine `11101100`.

**Sağlama:** $+20 + (-20)$**.**

```
  00010100
+ 11101100
----------
1 00000000
```

Soldan dışarı bir 1 çıkar. 8 bitlik sistemde çalıştığımız için bu bit dikkate alınmaz ve sonuç 0 olur. Herhangi bir işaret kontrolü yapmadık.

> **Ders sonrası not:** Hoca dışarı çıkan bu bite "overflow, taşma" dedi. Bu bit en soldaki elde (carry) bitidir. Sonuç olan 0, 8 bitin $-128$ ile $+127$ aralığı içindedir, yani sonuç bakımından bir taşma yoktur. İkiye tümleyende aritmetik taşma terimi büyük olasılıkla sonucun bu aralığın dışına çıktığı durumlar için kullanılır.

Hoca sonra 4 bitlik tabloyu kurdu. $0$'dan yukarı doğru birer artırarak $+7$'ye kadar gidilir. Bir sayı daha eklenemez, çünkü işaret biti 1 olur ve kural bozulur. Negatifler kısa yolla bulunur. Sıfıra uygulanırsa `0000`'da hiç 1 görülmediği için sayı kendisi kalır. Yani $+0$ ve $-0$ tek noktada birleşir.

| Bit dizisi | Değer | Bit dizisi | Değer |
|---|---|---|---|
| `0000` | $0$ | `1000` | $-8$ |
| `0001` | $+1$ | `1111` | $-1$ |
| `0010` | $+2$ | `1110` | $-2$ |
| `0011` | $+3$ | `1101` | $-3$ |
| `0100` | $+4$ | `1100` | $-4$ |
| `0101` | $+5$ | `1011` | $-5$ |
| `0110` | $+6$ | `1010` | $-6$ |
| `0111` | $+7$ | `1001` | $-7$ |

Geriye yazılmamış bir tek `1000` kalır. İşaret biti negatifliği gösterir. Bu dizi, işaret-büyüklükte boşa giden $-0$ yerine burada $-8$ olarak değerlendirilir. Böylece hiçbir bit dizisi boşa gitmez.

Bu yöntemin avantajları şöyle özetlendi:

- Dönüşüm kolaydır.
- Tek bir sıfır vardır, belirsizlik yoktur.
- İşaret kontrolü gerekmez.
- Bütün bit dizileri kullanılır, açıkta kalan yoktur.

### 1.5 Gösterim Aralığı

Hoca, $n$ bitli ikiye tümleyende aralığın slayttaki gibi olacağını, sınırların da dahil olduğunu söyledi. 10 bit örneğinde pozitif tarafta $+511$'e, negatif tarafta $-512$'ye kadar ulaşılır.

$$-2^{n-1} \leq x \leq 2^{n-1} - 1$$

**Ders sonrası not:** Hoca tabloyu kurarken sözlü olarak pozitif sınırı "$2^n - 1$", negatif sınırı "$-2^n$" dedi. Doğrusu yukarıdaki aralıktır. 4 bitte $-8$ ile $+7$, 10 bitte $-512$ ile $+511$ bulunur. Hocanın verdiği 10 bit sayıları bu aralıkla uyuşur.

## 2. Gerçel Sayılar ve IEEE 754

Tam sayılar her zaman işimize yaramaz. Günlük hayatta bazı şeyler hassasiyet gerektirir, sinyallerin çoğu da hassas değerler taşır. Bu yüzden gerçel sayılara ihtiyaç vardır.

### 2.1 Sabit ve Kayan Nokta

Gerçel sayı üç parçadan oluşur: işaret, tam kısım ve virgülden (uluslararası yazımda noktadan) sonraki ondalık kısım. Bu yazıma sabit noktalı gösterim (fixed point notation) denir.

Sayıyı bilgisayarda modellemek için ise başka bir kural gerekir, yoksa aynı sayı için pek çok yazım ortaya çıkar. Örneğin bir sayı virgülden önce 3 basamak olacak şekilde yazılabilir, üs ayarlanır ve değer değişmez. Matematiksel olarak fark yoktur, ama tam kısım ile ondalık kısmın saklanması farklıdır ve bu hassasiyet kaybı demektir. Bu yüzden bilimsel gösterim (scientific notation) kullanılır: Virgülden önce tek bir basamak bulunur, geri kalanlar virgülden sonra yazılır, sayı da uygun üsle çarpılır. Buna kayan noktalı gösterim (floating point) denir. Standart hâli IEEE 754'tür ve iki sürümü vardır. Bu gösterimle gerçel sayılar 0 ve 1'lerle yazılabilir.

Slaytta çok büyük bir sayıya örnek olarak $1 \times 10^{85}$ gösterildi.

### 2.2 25,75 Sayısının İkilik Yazımı

Hoca slaytta bu örnekte hatalar fark etti ve düzeltti: "10 üzeri 3" yerine $2^3$, "2 üzeri 1" yerine $2^0$ yazılmalıydı. Düzeltilmiş açılım şöyledir:

$$11001{,}11_2 = 2^4 + 2^3 + 2^0 + 2^{-1} + 2^{-2} = 16 + 8 + 1 + \tfrac{1}{2} + \tfrac{1}{4} = 25{,}75$$

Onluk sistemde virgülden sonra onda birler, yüzde birler varsa, ikilik sistemde ikide birler, dörtte birler, sekizde birler vardır.

Virgül kaydırılıp başta tek basamak bırakılınca bilimsel gösterime uygun hâle gelinir:

$$11001{,}11_2 = 1{,}100111_2 \times 2^4$$

Buradaki kesir kısmına mantis (mantissa), 4 değerine üs (exponent) denir.

### 2.3 Tek Duyarlık (32 Bit)

32 bitin dağılımı şöyledir:

| Alan | Bit sayısı |
|---|---|
| İşaret ($s$) | 1 |
| Üs ($E$) | 8 |

| Alan | Bit sayısı |
|---|---|
| Kesir ($F$, mantis) | 23 |

Sayı şu formülle hesaplanır:

$$N = (-1)^s \times 1{,}F \times 2^{E-127}$$

$s$ 1 ise sayı negatif, 0 ise pozitiftir. Üs alanının onluk karşılığından 127 çıkarılır ve 2'nin o kuvveti alınır. Hoca bunun bir normalleştirme olduğunu söyledi. Örneğin $2^0$ yazmak için üs alanına $0 + 127 = 127$ yazılır. Hoca önce alanları ters söyledi, sonra düzeltti: 8 bitlik alan üs, 23 bitlik alan mantistir.

> **Ders sonrası not:** Hoca 127'yi önce "$2^8 - 1 - 1$" diye söyledi, hemen ardından "$2^7 - 1$" diye düzeltti. Doğrusu $2^{8-1} - 1 = 127$'dir.

> **Ders sonrası not:** Hoca, "$2^{-127}$ için üs alanı `00000000` olur" dedi. Formülde $-127 + 127 = 0$ çıkar, ama üs alanının 0 olması aşağıda anlatılan sıfır ve normalleştirilmemiş sayı durumlarına ayrılmıştır. Hesapla bakıldığında tek duyarlıkta en küçük normal sayı $2^{-126} \approx 1{,}18 \times 10^{-38}$'dir.

### 2.4 Örnek: 25,75'in IEEE 754 Tek Duyarlık Karşılığı

- **Adım 1, ikiliğe çevirme:** $25 = 11001_2$, $0{,}75 = 0{,}11_2$.
- **Adım 2, normalleştirme:** Virgülün solunda yalnız bir tane 1 kalacak şekilde 4 bit kaydırılır: $1{,}100111 \times 2^4$. Sayı sıfıra yakınsa (0,75 gibi) kaydırma öbür yöne yapılır.
- **Adım 3, işaret:** Sayı pozitif olduğu için $s = 0$.
- **Adım 4, kesir:** `100111` yazılır, 23 bite kadar kalan yer 0 ile doldurulur: `10011100000000000000000`.
- **Adım 5, üs:** $4 + 127 = 131 = 10000011_2$.
- **Adım 6, birleştirme:** `0 10000011 10011100000000000000000`.

Bu 32 bit, 16'lık sistemde sağdan dörder dörder gruplanarak kısaca yazılır. Sağdan dört grup `0000`, sonra `1110` = E, `1100` = C, `0001` = 1, `0100` = 4 gelir (hexadecimal basamaklar 12 için C, 14 için E'dir). Sonuç: `0x41CE0000`. Baştaki `0x` sayının 16'lık olduğunu gösterir. Python ile doğrulandı.

> **Ek bilgi (derste anlatılmadı):** Normal sayılarda virgülün solundaki 1 her zaman bulunduğu için alanlara yazılmaz, formüldeki $1{,}F$ bu yüzden kullanılır. `0x41CE0000` Python ile çözüldüğünde $s = 0$, $E = 131$ alanlarından $(1 + F/2^{23}) \times 2^4 = 25{,}75$ değeri geri elde edilir. Yani saklanan yalnız `100111...` kısmıdır. Hocanın aşağıda sözünü ettiği "gizli bir" budur.

**Soru:** Bu örnekte 127'yi topladık, ama formülde çıkarılıyordu. Neden? **Cevap:** Formül, ikilik gösterimden sayının kendisine geçiştir, orada çıkarılır. Bu örnek tam tersi yöndedir, sayıdan ikilik gösterime gidiyoruz, onun için toplanır.

### 2.5 Özel Durumlar

Üs ve kesir alanlarının bazı değerleri özel anlamlıdır:

| Üs alanı | Kesir alanı | Anlamı |
|---|---|---|
| 255 | 0 | $\pm\infty$ (sonsuz) |
| 255 | $\neq 0$ | NaN (not a number) |
| 0 | 0 | Sıfır |
| 0 | $\neq 0$ | Normalleştirilmemiş (denormalized) sayı, gizli 1 yok |

Hoca, NaN'ı MATLAB ve Python'da çok göreceğimizi söyledi: Bir şey hesaplarsınız, sonuç sayı çıkmaz.

**Ders sonrası not:** Hoca NaN için "muhtemelen dışarıya taşma olmuştur" dedi. Python ile denendiğinde taşma ($10^{308} \times 10$) `inf` verir. NaN ise $0/0$ ve $\infty - \infty$ gibi tanımsız işlemlerden çıkar.

### 2.6 Çift Duyarlık (64 Bit)

Virgülden sonraki kısım bazen kendini sürekli tekrar eder (hocanın örnekleri $10/3$ ve $3/10$). Virgülden sonra ne kadar çok bit saklanırsa değer o kadar hassas tutulur. 64 bitlik sürümde üs 11 bit, kesir 52 bittir. Mantık aynıdır: Kesir ve üs kısımları bulunup bitlere yerleştirilir.

Sapma değeri 11 bitten gelir. Hoca bunu 1024'ten 1 çıkararak gösterdi: $2^{10} - 1 = 1023$.

$$N = (-1)^s \times 1{,}F \times 2^{E-1023}$$

| | Tek duyarlık | Çift duyarlık |
|---|---|---|
| Toplam bit | 32 | 64 |
| Üs biti | 8 | 11 |
| Kesir biti | 23 | 52 |
| Sapma | 127 | 1023 |
| Yaklaşık ondalık basamak | 7 | 15 |
| Yaklaşık aralık | $10^{\pm 38}$ | $10^{\pm 308}$ |

**Ders sonrası not:** Hoca çift duyarlığın aralığı için "$10^{\pm 306}$" dedi. numpy ile en büyük çift duyarlıklı sayı $1{,}80 \times 10^{308}$, tek duyarlıklı sayı $3{,}40 \times 10^{38}$ çıkar. Yani doğrusu $10^{\pm 308}$'dir. Hoca ayrıca "2 üzeri 11, eksi 1, eksi 1" diye başladı. Doğru yazım $2^{11-1} - 1$'dir.

## 3. İkili Kodlama: BCD, Gray, Eşlik, ASCII ve Unicode

Verileri sayısal bir sistemde saklamak için voltaj veya akım gibi elektriksel karşılıklara ihtiyaç vardır. Bunu analog olarak yapamayız. Örneğin 5 voltu A ile, 4,5 voltu B ile kodlasak kayıplar ve yanlışlıklar ortaya çıkar. İkili sistem ise bilgiyi voltajın varlığı veya yokluğuyla saklar. Bu bilgiyi bit kümeleri şeklinde sembolleyerek kodlama yaparız.

Veriler sayısal (numeric) veya sayısal olmayan (non-numeric) olabilir. Karakterler sayısal olmayan veriye örnektir: Üzerlerinde doğrudan aritmetik işlem yapılmaz, ama sayılar şeklinde saklanırlar.

$n$ bit ile $2^n$ farklı kod yazılabilir. Elimizde $N$ tane etiket varsa, gereken bit sayısı $\lceil \log_2 N \rceil$'dir, yani logaritma üste yuvarlanır. Slaytın örneklerinde 9 değer için $\lceil \log_2 9 \rceil = 4$ bit (hesap eklendi), 4 değer için 2 bit gerekir.

### 3.1 BCD ve Gray Kodu

Hoca bu kodların ayrıntısının lojik dersinin konusu olduğunu söyledi. 8421 kodu dediğimiz şey BCD'dir: Basamak ağırlıkları $2^3, 2^2, 2^1, 2^0$'dır. Sayılar birer artırılarak sırayla bir koda karşılık getirilir.

**Ders sonrası not:** Hoca ağırlıkları sayarken "2 üzeri 4" dedi. 8'in karşılığı $2^3$'tür.

Gray kodunda ise bir koddan sonrakine geçerken her seferinde yalnız 1 bit değişir. BCD'de böyle bir özellik yoktur: `0111`'e 1 eklenince `1000` olur ve 4 bitin hepsi değişir. Python ile 4 bitlik Gray dizisi üretilip ardışık kodlar arasında tek bit değiştiği doğrulandı.

Hoca slayttaki BCD örneğini de okudu: 57'yi saklamak için 5 ve 7 ayrı ayrı kodlanmış. "Bu pek geçerli değil, asıl yapacağımız 57'yi doğrudan dönüştürmek" dedi, ama bunun da bir dönüşüm olduğunu ve bazı yerlerde kullanıldığını ekledi.

**Ders sonrası not:** BCD büyük olasılıkla tam olarak bu anlama gelir, yani her ondalık rakam ayrı ayrı 4 bitle kodlanır: 57 = `0101 0111`. 57'nin doğrudan ikilik dönüşümü ise `111001`'dir. Hocanın "`1111`'e kadar sayma" anlatımı doğrudan ikilik sayımdır. BCD'de büyük olasılıkla yalnız `0000`-`1001` kullanılır.

### 3.2 Eşlik Denetimi (Parity)

Bir yerden bir yere veri aktarırken yapılan basit bir hata denetimidir. Giden bitlerdeki 1'lerin (veya 0'ların) sayısının tek mi çift mi olduğu kontrol edilir. Hem 1'lere hem 0'lara göre yapılabilir. Tek eşlikte (odd) teklik, çift eşlikte (even) çiftlik aranır.

Slaytta 3 bitlik veriler için tablo vardı. 1'ler üzerinden düşünürsek:

- `000` verisinde 1 sayısı 0'dır, yani çifttir. Çift eşlik biti 0, tek eşlik biti 1 olur.
- `111` verisinde 1 sayısı tektir. Çift eşlik biti 1 olur ve toplam `1111` gönderilir. Tek eşlikte bu bit 0'dır (hesap eklendi).

### 3.3 ASCII

ASCII (American Standard Code for Information Interchange), çok kullanılan sembollerin toplanıp belli bir sisteme oturtulmasıdır. Hoca bunu periyodik tabloya benzetti: Orada elementlerin özelliklerine göre bir düzen olduğu gibi burada da karakterlerin özelliklerine göre bir düzen vardır.

| Karakter | Onluk | İkilik | 16'lık |
|---|---|---|---|
| `0` | 48 | `00110000` | 30 |
| `9` | 57 | `00111001` | 39 |
| `A` | 65 | `01000001` | 41 |
| `a` | 97 | `01100001` | 61 |

Rakamlar 16'lık 30-39, yani onluk 48-57 aralığındadır ($48 + 9 = 57$). Büyük harften küçük harfe geçişte tek bir bit değişir, bu bitin değeri 32'dir. Hoca, Caps Lock veya Shift ile geçişin de bu işleme karşılık geldiğini söyledi. Tabloda 94 grafik karakter ve 34 basılı olmayan karakter vardır. Backspace gibi tuşlar ve bazı metin özellikleri ikinci gruptadır. Python ile sayıldı: $94 + 34 = 128$.

**Ders sonrası not:** Hoca 16'lık 41'in "64'e karşılık geldiğini" söyledi. Doğrusu $0x41 = 65$'tir, yani `A`'nın kodudur. Altyazıda `a` için geçen ikilik değer de bozuktur, doğrusu `01100001`'dir.

**Ders sonrası not:** Hoca "256 durum var, 8 biti de 1 yaparsak delete olur" dedi. 8 bitin hepsi 1 olan değer 255'tir. ASCII büyük olasılıkla 7 bitlik bir koddur ve delete karakteri 127, yani `01111111`'dir ($94 + 34 = 128$ karakter de 7 bite karşılık gelir).

### 3.4 Unicode

ASCII yetmeyince Unicode kullanılmaya başlanmıştır. Hoca bunun 16 bit olduğunu ve $2^{16} = 65\,536$ karakter sağladığını söyledi. Bu sayede bir Word dosyasında hem Türkçe hem Arapça hem Korece karakterler aynı anda yazılabilir.

**Ders sonrası not:** 16 bit ve 65 536 karakter büyük olasılıkla Unicode'un ilk sürümleri için geçerlidir. Bugün kod noktası sayısı büyük olasılıkla 1 114 112'dir ve UTF-8, UTF-16, UTF-32 gibi kodlamalarla yazılır.

Hoca ayrıca önceki derste konuşulan dönüşüm (conversion) ile kodlama (coding) ayrımını hatırlattı. Dönüşümde doğrudan matematiksel bir çıkarım vardır, kodlamada ise karşılıklar bir kuralla belirlenir.

## 4. Mantıksal İşlemler

Sayılar dışında, örneğin bir `if` koşulunda, doğru-yanlış (true-false) ifadeleri kullanırız. Bunlar matematiksel değil mantıksal ifadelerdir. AND, OR, XOR ve NOT işlemleri, hem işlemcide hem bellekte kullanılan mantık kapılarının temelidir. Hoca ayrıntıların ayrık matematik ve lojik devreler derslerinde görüleceğini söyledi. Analiz, bütün olası durumları listeleyen doğruluk tablolarıyla (truth table) yapılır.

- **NOT:** Giriş tersine döner. Gösterimi: NOT $A$, $\bar{A}$, $A'$.
- **AND:** Bütün girişler 1 ise sonuç 1'dir, en az bir 0 varsa sonuç 0'dır. Gösterimi: $A \cdot B$, $A \cap B$.
- **OR:** En az bir giriş 1 ise sonuç 1'dir, hepsi 0 ise 0'dır. Gösterimi: $A + B$, $A \cup B$. Buradaki artı matematikteki toplama değildir.
- **XOR:** Girişler aynıysa 0, farklıysa 1'dir. Özellikle kriptolojide çok kullanılır.
- **XNOR:** XOR'un tersidir. Girişler aynıysa 1, farklıysa 0'dır.

| $A$ | $B$ | AND | OR | XOR | XNOR |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 1 |

XOR, temel kapılarla şöyle açılır (doğruluk tablosu Python ile doğrulandı):

$$A \oplus B = A\bar{B} + \bar{A}B$$

Bu bölümle sayıların gösterimi kapanır ve hoca ara verir.

## 5. MATLAB'a Kısa Giriş ve Ek Kaynak

Aradan sonra hoca, online MATLAB'ın normal arayüzle aynı çalıştığını, dokümantasyonunun çok iyi olduğunu söyledi ve kısa bir gösteri yaptı.

- **Değişken:** `x = 5` yazınca değişken otomatik oluşur. Tipi tanımlanmaz, sayılar doğrudan double olarak tutulur. Harf atanınca değişken karakter olur.
- **Noktalı virgül:** Satır sonuna konmazsa sonuç ekrana yazılır, konursa yazılmaz.
- **Matris tabanlı dil:** Bütün değişkenler aslında matristir. Değişkene çift tıklayınca Excel tablosu gibi görünür.
- **Vektör:** `x = 1:5` birden beşe kadar değerleri vektör yapar.
- **Transpoz:** `x = x'` vektörün transpozunu alır.
- **Döngü ve diğerleri:** `for`, `if`, `printf` ve `scanf` benzeri giriş-çıkış işlemleri vardır. Örneğin yardım sayfasındaki `1:-0.2:0` döngüsü $1$'den $0$'a $0{,}2$ azalarak gider: $1;\ 0{,}8;\ 0{,}6;\ 0{,}4;\ 0{,}2;\ 0$ (numpy ile denendi).

  **Ders sonrası not:** Hoca "clear dediğimizde ekranı, clc dediğimizde ekranı siliyoruz" dedi. `clc` komut penceresini temizler, `clear` ise büyük olasılıkla çalışma alanındaki değişkenleri siler.

**Sembolik matematik.** `syms` ile değişken tanımlanıp denklemler kurulabilir. İleride göreceğimiz Laplace dönüşümü, bir düzlemden başka bir düzleme geçişin farklı bir hâlidir. Hoca dokümantasyondaki `laplace` örneğini okudu.

> **Ders sonrası not:** Altyazıya göre hoca "$1/x^2$'nin Laplace dönüşümü $\sqrt{\pi/s}$" dedi. Bu sonuç $1/\sqrt{t}$ içindir: $\mathcal{L}\{t^{-1/2}\} = \sqrt{\pi/s}$, sayısal integralle $s = 1, 2, 5$ için doğrulandı. $1/t^2$ için integral $t = 0$ civarında ıraksar: $\int_{\varepsilon}^{1} t^{-2}\,dt = 1/\varepsilon - 1$, $\varepsilon \to 0$ iken sınırsız büyür.

`int` ile integral de alınır. Hoca dokümantasyondaki $\sin(ax)\sin(x/a)$ fonksiyonunu gösterdi ve sonucun iki durumu olduğunu okudu. Bunun sebebi hesapla görülür: $a^2 = 1$ iken fonksiyon $\sin^2 x$ olur, $a^2 \neq 1$ iken integralde $a - 1/a$ paydası bulunur. Bu yüzden iki durum ayrı yazılır.

**Grafik.** `plot(x)` bir grafik penceresi açar. `x = 1:5` için sonuç düz bir grafiktir. Arayüzde birçok çizim türü seçilebilir. Hoca MATLAB programlama anlatmayacağını, ama sinyal çizimi gibi işlemleri örneklerle göstereceğini söyledi.

**Ek kaynak.** Hoca, *Schaum's Outlines* serisinin *Signals and Systems* kitabını önerdi. Kitapta konu anlatımı, 571 çözümlü örnek ve bundan fazla çözümsüz örnek vardır. Bu ders, bundan sonraki konular itibarıyla örnek çözmeyi gerektirdiği için pratik yapmak açısından önemlidir.

## 6. Sinyal ve Sistem Kavramı

### 6.1 Sinyal

Bir haberleşmeci veya bilişimci gözüyle sinyal denince radyo sinyalleri, kablo TV yayını, ses sinyali, bir devredeki voltaj ve akım akla gelir. Daha genel düşünürsek, ölçülebilen, bir şeyleri etkileyen veya bir şeyden etkilenen herhangi bir fiziksel veya soyut büyüklüğe sinyal diyoruz. Hocanın örnekleri şunlardır:

- Bisikletteki fren kablosundaki gerginlik
- Bir uzay aracının dönme oranı
- Bir hücredeki enzim yoğunluğu
- Euro cinsinden dolar fiyatı
- Bütçe açığı

Sinyal olmanın şartı, bir değişkene göre değişiyor olmaktır. "6 volt" doğrudan sinyal değildir, sabit bir büyüklüktür. Ama bir devredeki gerilim zamana göre değişiyorsa, o an değeri 6 volt olsa bile, bu bir sinyaldir. Değişken zaman olmak zorunda da değildir. Hocanın uydurma örneğinde, konum ilerledikçe binaların yüksekliği değişir ve bu bilgi de bir sinyal olarak saklanabilir. Sinyal birden fazla değişkene de bağlı olabilir.

### 6.2 Sistem

Sistem, bir sinyali alıp başka bir sinyale dönüştüren yapıdır. Radyo alıcısı, güçlendirici (amplifier), modem, mikrofon, telefon, hücre metabolizması ve küresel ekonomi birer sistemdir. Hepsinde bir giriş, bir çıkış ve arada gerçekleşen bir işlem vardır. Bir sistem içeride çok farklı tipte sinyaller barındırabilir ve birden çok giriş alabilir. Sistem bakışı, bunların hepsini bir arada düşünüp tasarım yapmayı sağlar.

Soyut olarak sistem, giriş sinyalini çıkış sinyaline dönüştüren bir blok şeklinde çizilir. Bu gösterim, sistemin fiziksel yapısından bağımsızdır.

**Yay örneği.** Bir yaya zamanla değişen, örneğin sinüzoidal, bir kuvvet uyguluyoruz. Giriş tek bir sayı değildir, zamana göre değişen bir sinyaldir. Yay kendine has karakteristiği olan sistemdir. Çıkışta da zamana göre değişen yeni bir sinyal olur. Bir öğrencinin hatırlatmasıyla formül $F = -kx$ olarak anıldı, yani $F(t) = -k\,x(t)$. Burada giriş de çıkış da bir sayı değil, bir dizi gibi düşünülmelidir.

**Su tankı örneği.** Bir musluktan $R_0(t)$ debisiyle su akar. Bu su $R_1(t)$ ile ikinci tanka, oradan $R_2(t)$ ile dışarı akar. $H_1$ gibi yükseklik parametreleri de zamanla değişir. Bütün bu büyüklükler sinyaldir. $R_0$ değiştiğinde $R_2$'nin davranışı ölçülmek istenir.

**Telefon örneği.** Konuştuğumuz yerde bir ses dalgası oluşur, sistem üzerinden taşınır, karşı tarafta konuştuğumuza yakın bir dalga elde edilmelidir. Çok farklı bir dalga çıkarsa ne söylediğimiz anlaşılmaz, gürültü duyulur.

### 6.3 Alt Sistemler

Sistemler modüler ve hiyerarşiktir. Telefon görüşmesinde ses en yakın baz istasyonuna, oradan öbürüne gider, arada kablolar olabilir ve sonunda karşı tarafa ulaşır. Yani giriş ile çıkış arasında alt sistemler vardır.

- **Art arda bağlı (cascade):** Bir alt sistemin çıkışı bir sonrakinin girişi olur.
- **Bileşik (composite):** Daha genel bir ifadedir. “Cep telefonu sistemi” derken içindeki ayrıntıları gizlediğimiz blok bileşik sistemdir.

### 6.4 Matematiksel Tanım

Matematiksel olarak sinyal bir fonksiyondur. Bağımsız değişken çoğunlukla zamandır, çünkü zamanla değişime odaklanırız. Bağımlı değişken ise voltaj, akış hızı veya ses basıncı gibi bir büyüklüktür. Hoca, ek slayttaki daha düzgün tanımı da okudu: Bir veya daha fazla değişkenin bilgisini aktaran olguya sinyal denir ve $n$ değişkenli bir sinyal şöyle gösterilir:

$$f(t_1, t_2, \dots, t_n)$$

İçerideki değişkenler bağımsız, sonuç bağımlıdır.

## 7. Sinyal ve Sistemlerin İlk Sınıflandırması

Sinyaller birkaç açıdan sınıflandırılabilir.

- **Bağımsız değişken sayısına göre:** Tek değişkenli veya çok değişkenli (multi-dimensional). Hocanın örneği resimdir.
- **Bağımsız değişkenin yapısına göre:** Zamanın her anında bilgi varsa sürekli, belli anlarda varsa ayrık. Bir dalgadaki voltaj süreklidir. Borsa endeksini günlere göre izlersek ayrıktır, çünkü günler sayılıdır.
- **Bağımlı değişkenin yapısına göre:** Sonuç sürekli ise analog, ayrık ise dijital.

  **Ders sonrası not:** Hoca burada resimde “hem zaman, hem x, hem y” değişimi olduğunu söyledi. Kendi sonraki anlatımına göre (Bölüm 14) sabit resim $x$ ve $y$'ye bağlı iki değişkenli sinyaldir, zaman eklenince video olur.

Sürekli sinyal, elimizi kaldırmadan çizilebilen sinyaldir. Ayrık sinyalde 0, 1, 2 anlarında birer değer vardır. Bazen ayrık sinyaller de aradaki değerler ara değerleme (interpolation) ile doldurularak sürekli gibi çizilir.

Sistemler ise şu açılardan ayrılır:

- **Giriş sayısına göre:** Tek girişli (single input) veya çok girişli (multiple input)
- **Çıkış sayısına göre:** Tek çıkışlı (single output) veya çok çıkışlı (multiple output)
- **İşlenen sinyalin boyutuna göre:** Tek boyutlu veya çok boyutlu
- **İşlenen sinyalin zamanına göre:** Sürekli zamanlı sistem sürekli zamanlı sinyalleri, ayrık zamanlı sistem ayrık zamanlı sinyalleri işler.

Slaytta iki blok şeması vardı:

- $x(t)$, sürekliden ayrığa dönüştürücüden geçer. Ayrık zamanlı sistem $p[n]$'den $q[n]$ üretir. Sonuç tekrar sürekli zamana çevrilir.
- $x[n]$ önce sürekli zamana çevrilir, sürekli zamanda işlenir, sonra tekrar ayrık olarak verilir.

Birinci yapıya hoca şu örneği verdi: Ses alınır, sayısala dönüştürülür, bilgisayarda bir algoritmayla işlenir. Sesi açmak aslında genliği yükseltmektir, yani `A = A*10` gibi bir katsayıyla çarpmaktır. Sonra sinyal yeniden sürekli hâle getirilip hoparlöre verilir. İkinci yapıda ise bilgisayardaki kayıtlı müzik önce sürekli işarete çevrilip bir devreye verilir. Aynı çarpma işlemi diyot ve kondansatör gibi devre elemanlarıyla yapılır, sonra sonuç yeniden ayrık hâle getirilir.

> **Hocanın vurgusu:** Ayrık zamanda köşeli parantez $x[n]$, sürekli zamanda yuvarlak parantez $x(t)$ kullanılacaktır.

## 8. Haberleşme ve Kontrol Sistemleri, Tacoma Köprüsü

**Haberleşme sistemi.** Mesaj sinyali vericiden (transmitter) geçer, taşınabilir bir sinyal hâline gelir, bir kanal sisteminden geçer ve alıcıya ulaşır. Burada geri besleme yoktur. Hoca, haberleşme öğrencilerinin bu dersin devamı olarak analog ve dijital haberleşme derslerini gördüğünü söyledi.

**Kontrol sistemi.** Örneğin bir ortamın sıcaklığını belli bir aralıkta tutmak isteyebiliriz. Bir referans giriş verilir, kontrolcü, plant denen kontrol edilen yapı üzerinde çalışır. Sensörler ve geri besleme (feedback) yardımıyla hata elde edilir. Sistem, hata en aza inene kadar çalışır.

Mühendisler olarak sinyalleri işlemek için sistemler tasarlarız. Bir yazılım sistemi de bunun örneğidir, ama burada kastedilen daha çok zamanla değişen bir yapının matematiksel modelini kurmaktır. Bu çerçeve, sistemin gereksinimleri karşılayıp karşılamadığını denetlemek için gerekir. Karşılamıyorsa sistem gözden geçirilir.

**Tacoma Narrows köprüsü.** Hocanın örneğinde köprü, rüzgârın tetiklediği titreşimler ve kararsız (unstable) mekanik yapı yüzünden yıkılmıştır. Kararlı (stable) ve kararsız sistemler ileride görülecektir. Yanlış tasarım burada bir felakete yol açmıştır.

## 9. Frekans Düzlemi Neden Var?

### 9.1 Ayrıştırma

Slaytta açısal frekansları $\omega_1, \omega_2, \omega_3$ ve genlikleri $0{,}1$, $0{,}7$, $0{,}2$ olan üç sinüzoidal sinyal ile bunların toplamı olan $f(t)$ vardı:

$$f(t) = 0{,}1\sin(\omega_1 t) + 0{,}7\sin(\omega_2 t) + 0{,}2\sin(\omega_3 t)$$

Bunları toplamak kolaydır: MATLAB'da üç diziyi toplamak gibidir. İşin güzel tarafı tersi sorudur: $f(t)$'den bileşenleri elde edebilir miyiz? Buna ayrıştırma (decomposition) denir ve çok işimize yarar.

### 9.2 Telefon Tuş Sesleri (DTMF)

Hoca telefonunun tuşlarına basarak seslerin tonlarının farklı olduğunu dinletti. Bu fark, frekansların farkıdır. Şöyle bir problem düşünelim: Bir ses dosyasında arka arkaya tuş sesleri ve aralarında boşluklar var. Hangi tuşlara basıldığını bulabilir miyiz?

Bir öğrenci, her tuşun tonunun farklı olduğunu ve bu tonun eşleştirilerek bulunacağını söyledi. Hoca "çoğunlukla doğru" diyerek süreci şöyle tamamladı:

- Boşluğa kadar olan parça alınır. Bu sonuçta bir sayı dizisidir, örneğin 500 sayı.
- Frekans düzlemine geçilir ve frekans bileşenleri bulunur.
- Bileşenler kod tablosuyla karşılaştırılır ve tuş belirlenir.
- Aynı işlem sonraki parçalar için tekrarlanır.

Bu yönteme DTMF (dual tone multi frequency) denir. Tuşlar bir frekans tablosunda kodlanmıştır ve her tuş iki sinüzoidin toplamıdır. Tablonun değerleri şöyledir: satır frekansları 697, 770, 852, 941 Hz, sütun frekansları 1209, 1336, 1477 Hz.

| Tuş | Düşük frekans (Hz) | Yüksek frekans (Hz) |
|---|---|---|
| 1 | 697 | 1209 |
| 2 | 697 | 1336 |
| 3 | 697 | 1477 |
| 9 | 852 | 1477 |

> **Ders sonrası not:** 3 tuşu için altyazıda "1697" geçiyor. Doğrusu 697 Hz'dir ve bu değer 1 ve 2 tuşlarının satırıyla da tutarlıdır.

Bileşenleri toplayıp sinyal oluşturmak kolaydır. Toplam verildiğinde onu parçalamak ise zordur. Zaman düzleminde topladığımız bileşenleri zaman düzleminde ayıramayız, ama frekans düzleminde ayırabiliriz.

**Soru:** İki sayıyı topladığımızda, örneğin $5+5=10$ için, $10$'un hangi sayıların toplamı olduğunu söyleyemeyiz. Burada nasıl söylüyoruz? **Cevap:** Burada iki sayı toplanmıyor. Sinüzoidal dalga bir sayı değil, zamana göre değişen bir sinyaldir. $\sin 40$ sabit bir değerdir, ama $\sin 40t$ bir sinyaldir. $t$ ile olan değişimden faydalanarak bunun bir $\sin 40t$ işareti olduğunu tespit edebiliriz. Bunu tespit edince frekansı da tespit etmiş oluruz.

**Soru:** Her tuş için iki sinyalin toplamı yerine kendine özgü tek bir sinyal kullanmak daha kolay olmaz mıydı? **Cevap:** Teorik olarak olabilir, ama hata artar. Telefon hatlarının belli bir frekans aralığı vardır, dolayısıyla frekanslar birbirinden çok uzak olamaz. Hocanın varsayımsal örneğinde aralık 100-500 Hz olsun ve tuşlar 100, 110, 120 Hz diye kodlansın. 110 Hz kesin bulunamayabilir, 109 çıkarsa yanlış tuşa gidilebilir. Bu yüzden iki tonla kodlanmıştır: Birinde hata olursa öbüründen giderilir. Ayrıca kolay çözülme her zaman tercih edilmez. Güvenliğin önemli olduğu sistemlerde daha çok bileşen bir araya getirilip taşıyıcı sinyal oluşturulur.

### 9.3 Baskın Bileşen ve Temel Karakteristik

$f(t)$'ye bakınca en çok $\omega_2$ bileşenine benzediği görülür, çünkü genliği en büyük olan odur. Ama her zaman bakarak bu bilgiyi almak mümkün değildir. Frekans düzlemine geçince sinyal daha sade gösterilir ve daha farklı bilgiler elde edilir. Sistem analizinde de faydası vardır: Sistemin kararlı olup olmadığı burada incelenebilir ve temel karakteristiği ortaya çıkar.

Bu, bilgisayar mühendislerinin çok çalıştığı bir konudur. Bir parmak izi görüntüsünün kişiye özel bir karakteristiği vardır. Bunu elde etmek için frekans veya uzamsal düzlemde analiz gerekir. Ses imzası da böyledir: Benim sesimle başkasının sesi uyuşmaz. Sesin içinden temel karakteristikler çıkarılır. Bu analiz hem zaman hem frekans düzleminde yapılabilir. Bu işe ayırt edici öznitelik çıkarma (distinctive feature extraction) denir ve yapay zeka ile makine öğrenmesinin ana bileşenidir.

### 9.4 Piyano Örneği

Slaytta yaklaşık 3,5 saniye süren bir piyano sesi gösterildi. Önce bir boşluk vardır, ses yükselir, tepe yapar, yavaş yavaş sönümlenir ve kaybolur. Hoca bu biçime "ADRS" dedi. Sesin bir saniyelik dilimi alınıp yakından gösterilmişti.

> **Ders sonrası not:** "ADRS" büyük olasılıkla ADSR'dir (attack, decay, sustain, release). Hoca bunu "protokol" diye andı, ama bu büyük olasılıkla bir ses zarfı adıdır.

Zaman düzleminde bu dilime bakarak sesin hangi frekanslara ait olduğunu doğrudan bulamayız. Do, re, mi gibi notalar farklı frekansların birleşimidir. Telefon tuşundaki gibidir, ama orada iki yerine belki 10-12 bileşen vardır. Bu analizle müzik türü belirleme gibi uygulamalar yapılabilir.

Frekans düzlemine geçince genlik-zaman grafiği yerine genlik spektrumu (frequency magnitude) elde edilir. Bu gösterim sinyalde hangi frekansların etkin olduğunu söyler. Slaytta düşük frekanslar baskındır, yüksek frekanslar giderek azalıp kaybolur. Bizim için önemli olan, belli bir aralıktaki frekanslardan karakteristik özellik çıkarmaktır. Hoca, yüksek frekanslı kısımlar için "detay veya gürültü" dedi: "Detay her zaman işe yaramaz, bazı durumlarda yarar. Gürültü hiçbir zaman işe yaramaz."

Frekans düzlemi simetriktir. Hocanın okuduğu örnekte 800 Hz'de genliği yaklaşık 38 olan bir bileşen, $A\cos(2\pi\cdot 800\,t)$ biçiminde bir dalga demektir. Sinüzoidal dalgaları genellikle kosinüsle göstereceğiz. Slaytın devamında negatif taraf atılıp 0-1500 Hz gösterilmişti. Yaklaşık 300, 400, 550, 600, 800 ve 1000 Hz'de bileşenler görülüyordu. Sinyal bu bileşenlerin toplamıdır:

$$A_1\cos(\omega_1 t) + A_2\cos(\omega_2 t) + \ldots$$

### 9.5 Doğrusal Sistemlerin Analizi

Doğrusal (linear) sistemler, sinüzoidal bileşenleri ayrılabildiği için frekans düzleminde kolayca analiz edilir. Slaytta arabanın bir "körp" karşısındaki davranışını tahmin etme örneği vardı. Hoca bunun "frene basma gibi bir şey galiba" olduğunu söyledi. Giriş bellidir, sistem ise diferansiyel denklemlerden oluşan araba dinamiğidir. Sistem frekans bileşenleriyle modellenirse çıkış rahatça bulunur.

> **Ders sonrası not:** Slayttaki sözcük büyük olasılıkla İngilizce *curb* (kaldırım kenarı) idi. Yani örnek büyük olasılıkla frene basma değil, aracın kaldırıma çarpma veya çıkma durumudur.

Diferansiyel denklem çözmek zordur. Hoca bunu denklemlerle karşılaştırdı: $x + 2 = 5$ gibi birinci dereceden bir denklem hemen çözülür. İkinci dereceden denklem böyle çözülmez. Beşinci dereceden sekiz bilinmeyenli bir denklem ise çok daha zordur. Türev içeren denklemler de bunun gibidir. Frekans düzleminde izlenen yol şöyledir:

- Girişin frekans düzlemindeki karşılığı alınır.
- Sistemin frekans düzlemindeki karşılığı alınır.
- Çıkışın frekans düzlemindeki karşılığı bulunur.
- Ters dönüşümle zaman düzlemine geçilir.

Üçüncü adım çok kolaydır. Zaman düzleminde yapılan işlem konvolüsyon (convolution) denen ve ileride göreceğimiz bir operatördür. Frekans düzleminde bunun yerine bildiğimiz çarpma gelir.

Bu sayede doğrusal sistemler rahat kontrol edilir. Uçağın otomatik olarak düz uçması, aracın sabit hızda gitmesi, bir odanın sıcaklığının sabit tutulması bunun örnekleridir. Hoca, kontrol alanındakilerin MATLAB ve Simulink ile sistemi modelleyip frekans düzlemi ve Laplace yöntemleriyle çözdüğünü söyledi.

## 10. Geri Beslemeli Hız Kontrolü ve Salınım

Bir arabanın hızını kontrol etmek isteyelim. Basit sistemde gaz girişi $x(t)$ zamanla değişir, çıkış da aracın hızıdır. Bunlar birer dizi gibi incelenebilir. Ama pedala basarken "60'a basacağım" diyemeyiz, çünkü pedal mekanik bir parçadır. Az mı çok mu basacağımızı bilmek için şu anki hızı öğrenmemiz, yani geri bildirim almamız gerekir.

Bu otomatikleştirilebilir. İstenen hız ile ölçülen hız arasındaki fark hata (error) olarak alınır. Örneğin 60 ile gitmek istiyoruz ve hız 50 ise $50 - 60 = -10$ gelir. Hatanın tersi kadar hız artışı istenir. Hız 70 olsaydı azaltma istenirdi. Hata belli bir katsayıyla çarpılıp gaz olarak sisteme geri verilirse sistem istenen hıza ulaşabilir. Sıcaklık kontrolü de aynıdır.

Fakat bu, kolayca salınıma (oscillation) yol açar. Sıcaklık referansa ulaşınca sistem durdurulur, sıcaklık düşmeye başlar, yeniden çalıştırılır, yine ulaşınca durdurulur. Aç-kapa (on-off) kontrolcüler böyle çalışır. Hocanın örneğinde sıcaklık 30'a varınca sistem kapatılır, 20'ye inince tekrar açılır ve sistem bu iki sınır arasında gidip gelir. Sürekli aç-kapa, devreyi yıpratır ve bozar. İyi bir kontrolcü hatayı azalta azalta sistemi istenen düzleme oturtur. Sistem, başka bir etki gelene kadar orada kalır. Böyle bir sistemin tasarım ve analizi frekans düzleminde çok kolaylaşır.

## 11. Kosinüs Sinyali ve Bileşenleri

> **Hocanın vurgusu:** Bundan sonraki sınıflandırmalar çok önemlidir ve dönem boyunca kullanılacaktır.

Hoca önce bir terim üzerinde durdu. Sürekli zamanlı ve ayrık zamanlı sinyaller slaytta "as the name suggests" ifadesiyle anlatılıyordu. *Suggest* çoğunlukla "önermek" diye bilinir, ama akademik metinlerde "öne sürmek, ortaya koymak, göstermek" anlamında kullanılır. Bu ifade "adından da anlaşılacağı üzere" demektir.

Sürekli zamanlı sinyaller zaman ekseni boyunca bütün gerçel sayıları içerir. Ayrık zamanlı sinyaller ise yalnız örneklenmiş değerleri içerir.

**Neden kosinüs?** Birim çemberde kosinüs ekseni üzerinden düşünülürse $\cos 0 = 1$, $\cos(\pi/2) = 0$, $\cos \pi = -1$'dir. Yani kosinüs $-1$ ile $+1$ arasında dolaşır. Hoca buna "doğal sinyal" dedi ve bunu teoride bütün sinyallerin kosinüs sinyallerinin toplamı olarak yazılabilmesiyle açıkladı.

> **Ders sonrası not:** Hoca "sinüs doğal değil, sonradan üretilmiş" dedi. Sinüs ve kosinüs aynı dalga biçimidir, aralarında $\pi/2$ faz farkı vardır: $\sin(\omega t) = \cos(\omega t - \pi/2)$ (numpy ile doğrulandı). Kosinüs tercihi büyük olasılıkla bir gelenektir, sinüsü dışlayan bir özellik değildir.

**Ek bilgi (derste anlatılmadı):** $A\cos(\omega t+\varphi)$, karmaşık üstel $Ae^{j(\omega t+\varphi)}$ ifadesinin gerçel kısmıdır. Bu örnek için numpy ile karşılaştırıldı, fark sıfır çıktı. Kosinüsle yazımın kolaylıklarından biri büyük olasılıkla bu bağlantıdır.

**Parametreler.** Tek bileşenli sinyal şöyle yazılır:

$$x(t) = A\cos(2\pi f t+\varphi) = A\cos(\omega t+\varphi)$$

$A$ genlik (amplitude), $f$ frekans, $\varphi$ fazdır. Açısal frekans $\omega = 2\pi f$, periyot $T = 1/f$'dir. Birden çok bileşen varsa her birinin kendi genliği, frekansı ve fazı bulunur:

$$x(t) = \sum_{i=1}^{N} A_i\cos(\omega_i t+\varphi_i)$$

**11.1 Örnek:** $x(t) = 3\cos(2\pi\cdot 2t - 3\pi/4)$

**Genlik.** Kosinüs $[-1,+1]$ aralığında olduğu için önüne gelen 3, sinyali $[-3,+3]$ aralığına çeker. Genlik $A = 3$'tür.

**Hocanın vurgusu:** Genlik maksimum değerdir, burada 3'tür. Geçen yılki bir vize sorusunda bazı öğrenciler 6 yazmıştı. 6, sinyalin aralığının (range) genişliğidir, genliği değildir.

**4 bitle kodlama.** Bu sinyal 4 bitle kodlanmak istense $2^4 = 16$ seviye olur. Aralık 6 olduğuna göre her zonun genişliği önceki derste konuşulan $\Delta$ kadardır:

$$\Delta = \frac{6}{16} = 0{,}375$$

Alttan üste doğru `0000`, `0001`, `0010`, ..., `1111` kodları birer $\Delta$ artışla dizilir.

**Ders sonrası not:** Hoca `0000`'ı alt sınıra, `1111`'i üst sınıra yazdı ve aralığı 6/16'ya böldü. 6/16 değeri, aralığın 16 eşit zona ayrılmasına karşılık gelir ve her kod bir zonu gösterir. Kodların kendileri tam olarak $-3$ ve $+3$ noktalarına konsaydı aralarında 15 adım olur, adım $6/15 = 0{,}4$ çıkardı.

**Frekans ve periyot.** $2\pi\cdot 2t$ ifadesi $2\pi f t$'den gelir. Yani $f = 2$ Hz'dir. Frekans, sinyalin kendini belli aralıklarla tekrar etmesidir. Slaytta bu, iki tepe noktası arasındaki süreden bulunuyordu. Sinyalin 3 değerine ulaştığı nokta için kosinüsün içi sıfır olmalıdır (ara adım eklendi):

$$4\pi t - \frac{3\pi}{4} = 0 \quad\Rightarrow\quad t = \frac{3}{16}\ \text{s}$$

Kosinüs periyodik olduğu için sonraki tepeler $3/16 + 1/2$, $3/16 + 1$, ... s anlarındadır (numpy ile bu anlarda değerin 3 olduğu doğrulandı). Demek ki işaret kendini $T = 1/2$ s ile tekrar eder. Bunun tersi $f = 1/T = 2$ Hz'dir. Açısal frekans $\omega = 2\pi f = 4\pi$ rad/s'dir.

**Faz.** Hoca fazı "$t = 0$ anında işaretin gösterdiği değer" diye anlattı. $t = 0$ yazınca birinci terim kaybolur ve $3\cos(-3\pi/4)$ kalır. Slaytta faz olarak $3\pi/4$ yazıyordu, işareti de sinyaldeki eksiden gelir.

**Ders sonrası not:** Faz, kosinüsün içindeki açı sabitidir: $\varphi = -3\pi/4$ rad. $t = 0$ anındaki genlik ayrı bir değerdir. Hoca $\cos(-3\pi/4)$ için "eksi bir bölü dört gibi bir şey" dedi. Doğrusu $\cos(-3\pi/4) = -\sqrt{2}/2 \approx -0{,}707$ ve $x(0) = -3\sqrt{2}/2 \approx -2{,}12$'dir.

Tek kosinüs bileşeni olan, hele denklemi verilmiş bir sinyalde bunları bulmak kolaydır. Asıl lazım olan, karmaşık bir dalganın içindeki kosinüs bileşenlerini bulmaktır. Bunu da frekans düzlemine geçerek yapacağız.

# 12. Sinyallerin Sınıflandırılması

## 12.1 Sürekli ve Ayrık Zamanlı Sinyaller

Zamanın bütün anlarında değer varsa sinyal sürekli, belli aralıklarda değer varsa ayrıktır. Burada notasyon önemlidir: Birinde değişken $t$, öbüründe $n$'dir.

> **Hocanın vurgusu:** Hangi düzlemde çalıştığınızı bilin. $x(t)$'den örnekleme ile $x[n]$'e, ya da $s$ veya $z$ düzlemine geçtiyseniz, artık orada $t$'ye dair bir şey kalmamalıdır. Bu hata çok yapılıyor. Yanlış düzleme geçmek soruyu baştan yanlış çözmek demektir. Geçiş yapıldıysa sonraki işlemler o düzlemde yapılır. Bütün işlemler bitince, gerekiyorsa ters dönüşümle $x(t)$'ye dönülür.

## 12.2 Analog ve Dijital Sinyaller

Analog sinyalde fonksiyonun alabileceği değerler süreklidir. Hoca bunu zamanda ve genlikte süreklilik olarak anlattı: Orijinal sinyali birebir kaydetmek gibidir. Dijital sinyalde ise değerler ayrıktır. 0 ve 1'lerden oluşan ikili diziler (binary sequence) dijital sinyallere karşılık gelir.

> **Ders sonrası not:** Hoca birkaç yerde sürekli-ayrık ile analog-dijital ayrımını aynı şey gibi söyledi. Kendi ilk sınıflandırmasına göre (Bölüm 7) sürekli-ayrık ayrımı zaman eksenine, analog-dijital ayrımı ise değerlere bakar. Örneklenmiş ama nicemlenmemiş bir dizi, ayrık zamanlıdır ve büyük olasılıkla tam anlamıyla dijital sayılmaz.

## 12.3 Periyodik ve Periyodik Olmayan Sinyaller

Periyodiklik, sinyalin belli bir noktadan sonra kendini tekrar etmesidir. Şart şudur: Her $t$ için

$$x(t+T) = x(t)$$

eşitliğini sağlayan bir $T$ değeri varsa sinyal periyodiktir. $t$ değişir, $T$ sabit kalır, eşitlik bütün $t$'ler için geçerli olmalıdır.

> **Ders sonrası not:** Hoca aperiyodik sinyal için "bu eşitliği sağlayan bir $T$ değeri buluyorsak aperiyodik diyeceğiz" dedi. Kendi tanımına göre doğrusu tersidir: Her $t$ için bu eşitliği sağlayan hiçbir $T$ yoksa sinyal aperiyodiktir.

## 12.4 Nedensel Sinyaller

Hoca nedensel (causal), nedensel karşıtı (anti-causal) ve nedensel olmayan (non-causal) ayrımını anlattı. Kelime, bir şeyin nedeni anlamındaki nedensellikle ilgilidir.

- **Nedensel:** Bütün negatif zamanlarda sıfırdır. Önceki zamanlara dair bilgisi yoktur.
- **Nedensel karşıtı:** Bunun zaman ekseninde tersidir, $f(t)$ yerine $f(-t)$ gibi. Bütün pozitif zamanlarda sıfırdır.
- **Nedensel olmayan:** Hem pozitif hem negatif zamanlarda değeri vardır.

> **Ders sonrası not:** Altyazıda "kaos" olarak geçen sözcük "chaos" olarak anlaşılmamalıdır. *Causal* büyük olasılıkla Latince *causa* (neden) kökünden gelir, İngilizcedeki *cause* ve *because* de böyledir.

Bu kavram ileride sistemler için de kullanılacaktır. Bir sistem yalnız şimdiki ve önceki zamanlara göre çalışıyorsa nedenseldir. Hoca önce $x[n] = x[n-1] + 2x[n]$ yazdı, sonra "$n-2$ diyelim" diyerek şuna getirdi: Dizinin şimdiki değeri, bir önceki değeri ile iki önceki değerinin iki katının toplamına eşittir.

$$x[n] = x[n-1] + 2x[n-2]$$

Bu nedensel bir sistemdir ve başlangıç değerleri de gerekir. Ayrıntısı ileride incelenecektir.

> **Ders sonrası not:** İlk yazılan $x[n] = x[n-1]+2x[n]$ düzenlenince $x[n] = -x[n-1]$ olur, yani hocanın anlattığı "iki önceki değer" ilişkisini vermez. Geçerli olan, düzeltilen $n-2$'li hâlidir.

### 12.5 Çift ve Tek Sinyaller

Tanımlar şöyledir:

- **Çift sinyal:** $f(t) = f(-t)$. Dikey eksene göre simetriktir.
- **Tek sinyal:** $f(-t) = -f(t)$.
- İkisi de sağlanmıyorsa sinyal ne tek ne çifttir.

Hoca sınıfa sinüs dalgasının çift olup olmadığını sordu. Bir öğrenci çift olamayacağını, negatif değerler yüzünden simetrinin bozulduğunu söyledi. Hoca sinyali iki yana periyodik çizdi ve $y$ eksenine göre simetri olmadığını gösterdi, yani sinüs çift değil, tektir. Kosinüs dalgası ise yukarıdan başlar. Bir periyot daha çizilince iki taraf birebir eşleşir, yani kosinüs çifttir.

> **Ders sonrası not:** Hoca tek sinyal için "$x$ eksenine göre simetri" dedi. Tanımdan görüldüğü gibi $f(-t) = -f(t)$ orijine göre simetridir.

**Tek-çift ayrışımı.** Herhangi bir sinyal şöyle yazılır:

$$f(t) = \underbrace{\tfrac{1}{2}\left[f(t) + f(-t)\right]}_{e(t)} + \underbrace{\tfrac{1}{2}\left[f(t) - f(-t)\right]}_{o(t)}$$

Sinyal çiftse $f(t) = f(-t)$ olduğu için ikinci terim gider, birinci terim $\frac{1}{2} \cdot 2f(t) = f(t)$ verir. Sinyal tekse birinci terim gider ve yalnız ikinci terim kalır.

> **Ders sonrası not:** Hoca tek sinyal durumunda sonucun "$-f(t)$" olduğunu söyledi. $f(-t) = -f(t)$ yazılınca $o(t) = \frac{1}{2}[f(t) + f(t)] = f(t)$ çıkar, yani formül sinyalin kendisini verir. Bu, sinüs için numpy ile de doğrulandı.

**Örnek:** $f(t) = 3t$**.** Eğimi 3 olan bir doğrudur: $t = 1$'de 3, $t = 2$'de 6 olur. Formül uygulanınca $e(t) = \frac{1}{2}(3t - 3t) = 0$ ve $o(t) = \frac{1}{2}(3t + 3t) = 3t$ bulunur (ara adım eklendi). Çift bileşen yoktur, sinyalin tamamı tek bileşendir.

**Örnek: Dalga biçimi verilen sinyal.** Slaytta sinyalin yalnız çizimi verilmişti. Fonksiyon çizimden okunur:

$$f(t) = \begin{cases} t + 1, & -1 \le t < 0 \\ 1, & 0 \le t \le 1 \\ 0, & \text{diğer} \end{cases}$$

- **Çift kısım.** $f(-t)$, $f(t)$'nin $y$ eksenine göre yansımasıdır. İkisi toplanınca, sabit olan 1'e bir doğru eklenir. $t = 0$'da $1+1 = 2$, $|t| = 1$'de $1+0 = 1$ olur. İkiye bölününce $0 < |t| \le 1$ için (kapalı biçim eklendi)

$$e(t) = 1 - \frac{|t|}{2}$$

- **Tek kısım.** Bu kez $f(-t)$ çıkarılır. Sabitten eğimli doğru çıkarılınca $0 < t \le 1$ için $2o(t) = 1 - (1 - t) = t$ olur. Sol taraf da simetrik olarak negatiftir. Yani $|t| \le 1$ için (kapalı biçim eklendi)

$$o(t) = \frac{t}{2}$$

- **Sağlama.** Hoca iki kısmı toplayarak orijinal dalgayı buldu. $0 < t \le 1$ için $e(t) + o(t) = 1 - t/2 + t/2 = 1$, $-1 \le t < 0$ için $1 + t/2 + t/2 = t + 1$ olur (ara adım eklendi). numpy ile tüm aralıkta karşılaştırıldı, $e$'nin çift, $o$'nun tek olduğu ve toplamlarının $f$'yi verdiği doğrulandı.

$t = 0$ noktasında $f(0) = 1$ olduğu için $e(0) = 1$, $o(0) = 0$'dır. Hocanın anlattığı $2e(t)$ grafiğindeki 2 değeri bu noktaya sağdan ve soldan yaklaşılan değerdir.

### 12.6 Deterministik ve Rastgele Sinyaller

Deterministik sinyal, her durumda aynı sonucu veren, değerleri bir fonksiyonla kesin olarak belli olan sinyaldir. Hocanın örneği şudur:

$$x(t) = 2\cos(2\pi \cdot 200\, t + 500)$$

$t$ yerine ne zaman 100 verilirse verilsin aynı sonuç çıkar. 101 için farklı bir değer çıkar, ama o da her seferinde aynıdır.

Rastgele (random) sinyaller ise kesin (strict) olarak tanımlanmamıştır. Değerler data olarak kaydedilmiş olabilir, ama doğrudan bir fonksiyon yoktur. Araya değer vermek gerekirse ancak ara değerlemeyle yaklaşık değer bulunur. Her defasında farklı sonuçlar çıkabilir.

## 13. Örnek: Bir Sinyalin Özellikleri ve MATLAB ile Çizimi

### 13.1 Özelliklerin Belirlenmesi

Slaytta şu sinyalin özellikleri soruldu:

$$f(t) = \begin{cases} \dfrac{\sin(2\pi t)}{t}, & t \geq 1 \\ 0, & t < 1 \end{cases}$$

- **Sürekli:** $t$'ye bağlı değişir.
- **Analog:** Sürekli olduğu için analog diyebiliriz.
- **Periyodik değil:** Aperiyodiktir.
- **Sonsuz uzunlukta:** $t$ için üst limit konmamıştır.
- **Nedensel:** Sıfırdan küçük zamanlarda değeri yoktur.
- **Tek-çift:** Hoca, $f(t) = f(-t)$ ve $f(-t) = -f(t)$ şartlarının yerine yazılıp denetleneceğini söyledi. Karar kısmı derste söylenmedi. $t \geq 1$'de $f(t) \neq 0$ olan her noktada $f(-t) = 0$ olduğu için iki şart da sağlanmaz. Sinyal ne tek ne çifttir (ara adım eklendi).
- **Deterministik:** Bir fonksiyonla tanımlıdır, verilen her $t$ için sonuç kesindir.

  **Ders sonrası not:** Hoca periyodik olmamasının sebebini "sinüzoidal dalgaların hiç-birisi periyodik olmaz" diye açıkladı. Bu doğru değildir: Bir önceki örnekte kosinüsün periyodik olduğu gösterilmişti ve Bölüm 12.3'teki eşitliği sağlar. Bu sinyal, $t < 1$ için sıfır olduğu ve genliği $1/t$ ile azaldığı için periyodik değildir.

### 13.2 MATLAB Kodu

Kod, kısa nottaki hâliyle ve hocanın anlatımıyla uyumlu olarak şöyledir:

```
clc
clear all
close all

t1 = 1:0.01:10;
t2 = -10:0.01:1-0.01;
timeAxis = [t2 t1];
MySignal = [zeros(1,length(t2)) sin(2*pi*t1)./t1];
plot(timeAxis,MySignal)
ylabel('Amplitude', 'fontsize', 20)
xlabel('time', 'fontsize', 20)
title('My First Signal','fontsize', 20)
```

Bir öğrenci kodu yeniden anlatmasını isteyince hoca kodu MATLAB'a kopyalayıp satır satır çalıştırdı. Anlatım şöyledir:

- `t1` ve `t2` döngü yazmadan, MATLAB'ın hazır operatörüyle oluşturulur. `t1`, 1'den 10'a $0{,}01$ artarak gider ve 901 elemanlıdır.
- `t2`, $-10$'dan $0{,}99$'a $0{,}01$ artarak gider.

- `[t2 t1]` iki vektörü birleştirir (concatenation). Sinyalin $t < 1$ ve $t \geq 1$ kısımları farklı çalıştığı için zaman ekseni iki parçadan kurulur.
- `zeros(1,length(t2))` içi sıfır olan, `t2`'nin uzunluğunda bir dizi oluşturur. Hoca burada uzunluğu "500 tane" diye örnek olarak andı.
- `./` eleman eleman bölmedir. Her sinüs değeri, kendisine karşılık gelen $t$ değerine bölünür.
- `MySignal`'ın uzunluğu `timeAxis`'in uzunluğuna eşit olur. Değişken için ayrıca tanım yapılmaz.
- `plot` sinyali zaman eksenine göre çizer. `ylabel`, `xlabel`, `title` etiketleri yazar, yazı boyutu 20'dir.

Sonuçta belli bir yere kadar sıfır, sonra sönümlü sinüzoidal dalga görülür.

Kod Python'a (numpy) çevrilerek çalıştırıldı: `t1` 901, `t2` 1100, birleşik zaman ekseni ve sinyal 2001 elemanlıdır. `t2` son değeri $0{,}99$'dur. $t < 1$ için bütün değerler sıfır çıkmıştır.

> **Ders sonrası not:** `length(t2)` gerçekte 1100'dür. Hocanın "500 tane" ifadesi yalnız örnekti. Hoca ayrıca `./` için "dot product veya cross product diyoruz ya, burada da aynı durum var" dedi. `./` eleman eleman işlemdir, sonuç yine vektördür. Dot product ise büyük olasılıkla ayrı bir işlem olarak ele alınmalıdır.

## 14. Zaman ve Uzay Fonksiyonu Olarak Sinyaller, Sistem Türleri

Hoca ifade biçimine de dikkat çekti: Sürekli sinyal, ayrık sinyal yerine "sürekli zamanlı, ayrık zamanlı" diyoruz, çünkü konuşacağımız sinyaller genellikle zamanın fonksiyonudur. Dersin ana amacı da konuyu sürekli zaman ve ayrık zaman diye iki parçada yürütmektir.

Sinyaller uzayın fonksiyonu da olabilir. Görüntü iki değişkenlidir ve matris olarak saklanır: $x$ ve $y$ ekseni, her eleman bir pikseldir. Buna zaman eklenince video olur. Her boyut sürekli veya ayrık olabilir. Hocanın örneğinde yoldaki sabit bir ağaç iki boyutlu renk bilgisidir, rüzgârla hareket etmeye başlayınca zamana göre üç boyutlu bir sinyal olur. Ağacın fotoğrafını alınca bunu iki boyutlu ayrık sinyale, videosunu alınca üç boyutlu ayrık sinyale çevirmiş oluruz.

Sistemler giriş ve çıkış tipine göre üçe ayrılır:

- **Sürekli zamanlı sistem:** Girişi ve çıkışı sürekli sinyallerdir. Örnekler: FM ve AM radyo, mekanik araçlar.
- **Ayrık zamanlı sistem:** Girişi ve çıkışı ayrık sinyallerdir. Örnekler: bilgisayar oyunu, MATLAB, mortgage gibi kredi hesapları.
- **Hibrit sistem:** Genellikle analogdan sayısala (ADC) ve sayısaldan analoğa (DAC) dönüştürücüler kullanılır. Yani girişi sürekli alıp ayrığa çevirir veya tersini yapar.

Hibrit sistem örnekleri şunlardır:

- Bilgisayar oyununda ses komutu veya joystick girişi, sürekliden ayrığa dönüşüm içerir.
- Modern elektrikli araçlarda yol sürtünmesi bilgisi verilir.
- Araçta dış sıcaklık ve hız göstergede görünür. Sürekli sinyal alınır, ayrığa çevrilir, işlenip gösterilir. Gösterilen değer gerçek değerin kendisi değildir, belli bir ölçme hatası içerir.
- Ticari ve askeri uçaklarda hibrit sistemler kullanılır, çünkü bazı durumlarda sürekli yapı gerekir, bazı durumlarda ayrık.

## 15. Notasyon ve Örnekleme

### 15.1 Sürekli Zamanda Notasyon

Sürekli zamanlı sinyallerde genellikle $t$, $t_0$, $t_1$, $t_2$ gibi parametreler kullanılır. Fonksiyon $v$ diye gösterilebilir, bu durumda neye göre değiştiği yazılmaz ve zaman kabul edilir. $v(\cdot)$ veya $v(t)$ de yazılabilir. Parantez içine sayı konursa, örneğin $v(1{,}2)$, bu sinyal değil o andaki sabit bir değerdir.

Hoca ultrason cihazını da örnek olarak andı. Cihaz ses darbeleri gönderir, yansımadan görüntü elde edilir. Slaytta böyle bir darbe sinyalinin zamana göre değişimi vardı.

### 15.2 Ayrık Zamanda Notasyon

Ayrık zamanlı sinyal bir örnek dizisidir, C'deki bildiğimiz dizi gibi. $n$ tam sayıdır ve $x[n]$ biçiminde köşeli parantezle yazılır.

> **Hocanın vurgusu:** Parantez türüne dikkat edin. Yuvarlak ve köşeli parantezin karıştırılması sınavlarda sık yapılan bir hatadır.

### 15.3 Örnekleme

Ayrık sinyal sıklıkla sürekli işarette $t$ yerine $nT_s$ yazılarak elde edilir. $T_s$ örnekleme periyodudur. $T_s = 1/f_s$ olduğu için $n/f_s$ de yazılabilir:

$$x[n] = x(nT_s) = x\left(\frac{n}{f_s}\right)$$

$n$ yalnız tam sayı değerler alır ($0, 1, 2, \ldots$ ve $-1, -2, \ldots$). $-1{,}5$ gibi değerler olmaz.

**Örnek 1:** $x(t) = 5t$**,** $f_s = 500$ **Hz.**

- $T_s = 1/500$ s.
- $t$ yerine $n \cdot \frac{1}{500}$ yazılır: $x[n] = 5 \cdot \dfrac{n}{500}$.
- Sonuç: $x[n] = \dfrac{n}{100}$.

**Örnek 2:** $x(t) = \cos(2\pi \cdot 500\, t)$**.** Hoca sınıfa sorarak ilerledi.

- Frekans: $f = 500$ Hz.
- Açısal frekans: $\omega = 2\pi f = 1000\pi$ rad/s.
- Genlik: 1, çünkü katsayı 1'dir.
- Faz: 0. Bir öğrenci önce "$T$" dedi, hoca faz olması için $\omega t$'ye bir şey eklenmesi gerektiğini ve burada hiçbir şey olmadığını söyledi.
- En düşük örnekleme frekansı: $f_s > 2f_{max} = 2 \times 500 = 1000$ Hz. Bir öğrenci de "bin, iki katı" diye cevap verdi.

$f_s = 1000$ Hz ile örneklenirse:

- $T_s = 1/1000$ s.
- $x[n] = \cos\left(2\pi \cdot 500 \cdot \dfrac{n}{1000}\right) = \cos(\pi n)$.
- Yani $n$ tam sayıları için sonuç $x[n] = (-1)^n$ dizisidir: $1, -1, 1, -1, \ldots$ (bu son adım eklendi, numpy ile doğrulandı).

  **Ders sonrası not:** Hoca "en az iki katı" dedi, hemen ardından $f_s$ için "must be greater than" ifadesini kullandı. Doğru şart kesin eşitsizliktir: $f_s > 2f_{max}$. Tam $2f$ ile örneklemede sorun çıkabilir. Bu frekansta bir kosinüs $(-1)^n$ verirken aynı frekanstaki sinüs $\sin(\pi n) = 0$ verir ve bütün bilgi kaybolur (numpy ile doğrulandı).

## 16. Sinyal İşlemleri: Genlikte ve Zamanda Ölçekleme

Hoca bu işlemlerin önemli olduğunu söyleyerek konuya geçti.

### 16.1 Genlikte Ölçekleme

Genlikte ölçekleme, müziğin sesini açmaktır. Sinyal bir sayıyla çarpılır, yani dizideki bütün değerler o sayıyla çarpılır. Bu hem sürekli hem ayrık zamanda yapılabilir.

$$y(t) = a\,x(t), \qquad y[n] = a\,x[n]$$

- $a > 1$ ise değerler büyür, ses açılır. Örneğin $a = 2$ ile 1 olan örnek 2'ye çıkar.
- $0 < a < 1$ ise değerler küçülür, ses kısılır. Örneğin $a = 1/2$.

Zaman ekseninde değişiklik olmaz, yalnız genlik değerleri değişir.

### 16.2 Sürekli Zamanda Zaman Ölçekleme

Zaman değişkeni yerine $bt$ yazılır: $x(bt)$. Davranış $b$'nin değerine göre değişir.

- $b > 1$**, sıkıştırma (compression):** $x(2t)$ için, $x(t)$'nin $t = 1$'deki değeri artık $t = 0{,}5$'te bulunur.
- $0 < b < 1$**, germe (stretching):** $x(t/2)$ ile sinyal uzar.

Burada genlik değerleri değişmez, sinyalin başlangıç ve bitiş noktaları değişir.

**Soru:** Zaman ölçekleme nerede uygulanır? **Cevap:** Hocanın örneği, YouTube veya ders kaydını 1,5 ya da 2 hızda dinlemektir. $N$ örnek varsa, 2 ile çarpıldığında bunlar $N/2$ örnek üzerinden dinlenir. Arada kayıplar olduğu için çok hızlı konuşmada anlaşılmayan yerler olur.

Hoca bunu müzik parçası çalarak denedi. İnsan sesi hızlandırılınca anlaşılır kalır. Piyanoda kayıp az hissedilir, çünkü temiz bir sestir. Diğer enstrümanlar girince kayıp artar, bazı enstrümanlarda doğrudan cızırtı duyulur. Yavaşlatınca da orijinalden uzaklaşılır, çünkü araya ara değerlemeyle değerler eklenir.

> **Ders sonrası not:** Oynatıcılardaki hız ayarı büyük olasılıkla basit bir aşağı örnekleme değildir. Ses perdesini korumak için büyük olasılıkla zaman sıkıştırma ve germe algoritmaları kullanılır. Hocanın anlatımı sezgi içindir.

### 16.3 Ayrık Zamanda Aşağı Örnekleme

$x[2n]$ örneklerin bir kısmının atılması demektir.

- Sıfırdaki örnek yerinde kalır.
- $n = 1$'deki örnek gider, $n = 2$'deki örnek $n = 1$'e gelir.
- $n = 3$'teki örnek gider, $n = 4$'teki örnek $n = 2$'ye gelir.
- $n = -1$'deki örnek gider, $n = -2$'deki örnek $n = -1$'e gelir.

Buna aşağı örnekleme (downsampling) denir. Hoca, YouTube'da hızlandırmanın da bir anlamda bu olduğunu söyledi.

### 16.4 Ayrık Zamanda Yukarı Örnekleme

$x[n/2]$ için sıfırdaki örnek yerinde kalır, $n = 1$'deki örnek $n = 2$'ye gider. Bu durumda $n = 1$'e bir değer gerekir. Bu değer ya sinyalin denkleminden ya da ara değerlemeyle bulunur. Buna yukarı örnekleme (upsampling) denir.

**Soru:** 0 ile 1 arasına ortalama bir değer mi ekliyoruz? **Cevap:** Ortalama değil. Hocanın örneği $x[n] = 5n + 3$'tür. $x[n/2]$ yapılınca $n$ yerine $0{,}5$ gibi bir değer gelir ve onun değeri o noktaya yazılır. Bu örnekte $5(0{,}5) + 3 = 5{,}5$ bulunur (hesap eklendi).

> **Ders sonrası not:** Bu doğrusal örnekte $x[0] = 3$ ve $x[1] = 8$'in ortalaması da $5{,}5$'tir, yani bu sinyalde iki yol aynı sonucu verir. Hocanın "ortalama değil" demesi, kuralın ortalama almak olmadığı anlamındadır. Sinyal doğrusal değilse sonuçlar ayrılabilir. Ayrıca $n/2$ yalnız çift $n$ için tam sayıdır. Tek $n$'ler için büyük olasılıkla standart tanımda 0 yazılır ve aralar sonra ara değerleme ile doldurulur.

**Soru:** Araya yeni değer eklenmiyor da yalnız örnekler arasındaki periyot mu kısalıyor? **Cevap:** Ayrık zamanda böyle bir şey yoktur. Örnek sayısını artırmak gerekir, araya yeni örnekler gelir. Yukarı örneklemenin mantığı budur.

Hoca görüntü örneği de verdi. Görüntüye yaklaşmak veya uzaklaşmak aynı ölçeklemenin iki boyutlu hâlidir. Yaklaştıkça araya örnekler eklenir, uzaklaştıkça örnekler çıkarılır. Bunu yapan yine ara değerlemedir. Yaklaşınca bozulmalar görülebilir, ama vektörel resim formatlarında bozulma olmaz.

> **Ders sonrası not:** Hoca vektörel formatta bozulma olmamasını "çok fazla yeni eleman eklenmesine" bağladı. Büyük olasılıkla sebep, görüntünün piksel olarak değil geometrik tanımlarla saklanması ve her yakınlaştırmada yeniden çizilmesidir.

**Soru:** Sesi kaydederken çok fazla örnek alınması gibi bir durum var mı? **Cevap:** Bu, kayıt cihazının örnek alma kabiliyetiyle ve frekansıyla ilgilidir. Bazı cihazlar çok iyi kayıt alır, bazıları

daha az hassastır. Videonun sesin önünde veya arkasında kalması ise başka bir konudur, video ve sesin örnekleme frekanslarının uyuşmamasıyla ilgilidir.

## Dersin Toparlaması

Dersin ilk kısmında sayıların bilgisayarda gösterimi tamamlandı. İşaret-büyüklük ve bire tümleyende iki sıfır ve toplama sorunu görüldü. İkiye tümleyen tek sıfırı ve işaret kontrolü gerektirmemesiyle günümüzün standardı olarak anlatıldı. Gerçel sayılar için IEEE 754'ün tek ve çift duyarlık biçimleri, 25,75 örneği üzerinden işlendi. Ardından BCD, Gray, eşlik, ASCII, Unicode ve mantıksal işlemlere kısaca değinildi, MATLAB tanıtıldı.

İkinci kısımda dersin asıl konusuna geçildi. Sinyal, bir değişkene göre değişen büyüklük olarak, sistem de sinyali başka bir sinyale dönüştüren yapı olarak tanımlandı. DTMF, piyano ve araç dinamiği örnekleri, frekans düzlemine neden geçildiğini gösterdi: Bileşenler ayrışır, zaman düzlemindeki konvolüsyon çarpmaya dönüşür. Sonra sinyaller sürekli-ayrık, analog-dijital, periyodik-aperiyodik, nedensel, tek-çift ve deterministik-rastgele olarak sınıflandırıldı. Örnekleme ile $x[n] = x(nT_s)$ elde edildi, genlikte ve zamanda ölçekleme ile aşağı ve yukarı örnekleme görüldü. Hoca sinyal ve sistem özelliklerine öğleden sonra devam edileceğini söyledi. Bu sınıflandırmalar dönem boyunca kullanılacaktır. Stable-unstable sistemler, konvolüsyon, Laplace dönüşümü ve sistem nedenselliği ise sonraki konulardır.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.